

INTELLIGENT AML

Master Project Blueprint & Execution Plan v2.6

A Neuro-Symbolic Framework for Omni-Channel Financial Fraud Detection
via Temporal Graph Networks & Agentic AI

Prepared for Institutional Research & Enterprise Development

Executive Summary

Financial crime is among the most complex challenges facing modern economies, with legacy anti-money laundering (AML) systems suffering from "**Relational Blindness**". Traditional tabular ML systems fail to perceive multi-hop topological structures like *Smurfing* and *Peel Chaining*, resulting in unsustainable >95% false-positive rates.

The **Intelligent AML** framework resolves this by deploying a Heterogeneous Temporal Graph Neural Network (HT-GNN) combined with a Burst-Aware Edge Attenuation mechanism. This pipeline is fortified with a GraphGAN for minority class synthesis, Conformal Prediction for causal auditing, and an autonomous LangChain/CrewAI multi-agent system to automatically draft compliance-ready Suspicious Activity Reports (SARs) in under 60 seconds.

The 6-Layer Architecture Stack

The system divides responsibility into a Python-based orchestration plane and a C++/Rust computation plane to ensure low-latency processing, even on constrained hardware architectures (e.g., Core i5, 16GB RAM).

L1: Universal Ingestion Engine

- **Purpose:** Streams and sanitizes massive chronological transaction data without triggering local Out-Of-Memory (OOM) crashes.
- **Tech Stack:** Polars (Rust) + DuckDB (C++).
- **Mechanism:** Converts raw 125GB CSV historical datasets into highly compressed columnar Parquet formats, enabling zero-copy memory mapping.

L2: Deep Learning GAN (Data Augmentation)

- **Purpose:** Resolves the severe 97:3 class imbalance between legitimate and illicit financial transactions.
- **Tech Stack:** Wasserstein GraphGAN.
- **Mechanism:** Generates structurally valid synthetic money-laundering subgraphs to adequately train the detection engine on edge-case typologies.

L3: Core Detection & Federated Privacy

- **Purpose:** Identifies high-velocity smurfing and multi-hop peel chains across institutions without moving raw data.
- **Tech Stack:** `PyTorch Geometric (HT-GNN)` + `Flower (Federated Learning)` + `Opacus (Differential Privacy)`.
- **Mechanism:** Trains shared graph models directly inside localized bank firewalls, mathematically shielding consumer PII.

L4: Causal Auditing (Explainable AI)

- **Purpose:** Removes the "Black-Box" liability to satisfy regulatory laws like the EU AI Act and BFIU guidelines.
- **Tech Stack:** `GNNExplainer` + `Captum` + `PyVis`.
- **Mechanism:** Masks safe nodes to output high-fidelity visual heatmaps of the exact transaction pathways triggering the AML alerts.

L5: Agentic Automation

- **Purpose:** Automates the manual 4-6 hour reporting workload burdening human compliance officers.
- **Tech Stack:** `LangChain` + `CrewAI` + `ReportLab`.
- **Mechanism:** An autonomous LLM pipeline scrapes audit evidence from L4 to auto-populate unalterable PDF Suspicious Activity Reports (SARs).

L6: Application UI & Microservices

- **Purpose:** The human-in-the-loop validation command center.
- **Tech Stack:** `FastAPI` + `Streamlit`.
- **Mechanism:** Interactive 3D topology tracking, real-time alert queues, and threshold tuning arrays exposed via REST endpoints.

Hybrid Development Environment Protocol

Given the hardware constraints of the development machine (Core i5 8th Gen, 16GB RAM, H310 Motherboard), attempting to load graph models in-memory natively will result in catastrophic system failure. The project utilizes a **Hybrid Remote-Execution Strategy**:

- **The Control Plane (Local PC):** Handled via lightweight IDEs (Cursor / VS Code). Used exclusively for editing orchestration scripts, API definitions, and UI components. Memory footprint capped at ~4GB.
- **The Execution Plane (Kaggle/Cloud):** Heavy PyTorch Geometric matrix multiplication, NeighborLoader batching, and GraphGAN training are executed remotely on free-tier cloud GPUs (T4/P100), passing only aggregated model weights back to the local device.

Omni-Channel Dataset Portfolio

The system is trained on a comprehensive matrix of 11 multi-sector datasets designed to generalize model behavior across traditional banking, mobile financial services (MFS), and cryptocurrency networks.

Phase Grouping	Dataset Source	Sector / Graph Typology	Core ML Objective
Group A: Native	DS1: Elliptic Bitcoin DS2/10: Elliptic2 (KDD 24)	Crypto / Dynamic Trans. Graph	Detecting multi-hop peel chains & ransomware mixing.
Group A: Native	DS3: PaySim (PaySim1)	MFS / Synthetic Tabular Logs	Mapping high-velocity P2P cash-out bursts (e.g., bKash).
Group B: External	DS4/11: IBM AMLSim	Banking / Multi-Agent Ledger	Analyzing fiat trade-based layering flows.
Group B: External	DS5: SynthAML (Spar Nord)	Banking / Institutional Alerts	Establishing baseline institutional alert thresholds.
Group C: Scale	DS6: DGraph-Fin	Graph Proxy / Finvolution	Testing Out-Of-Core NeighborLoader scalability.
Group C: Scale	DS7: XBlock-ETH	Crypto / Smart Contracts	Mapping heterogeneous node relationships.

24-Week Tactical Roadmap & Metric Matrix

A sequential execution plan engineered to guarantee academic defense and commercial demonstrability.

Phase	Core Objective	Primary Metric Target	Contingency Fallback
Phase 1: Data & Benchmarking	Establish static GCN baseline on ingested Parquet files.	Document baseline Macro F1 & PR-AUC.	N/A (Standard Data Baseline Verification).
Phase 2: Graph Synthesis	Generate minority class nodes via Wasserstein GraphGAN.	MMD < 0.05 and GED ≤ 3.0	If unconverged by Wk 8: Drop GraphGAN, initiate SMOGN.
Phase 3: Federated Learning	Implement Flower logic with Differential Privacy.	Epsilon ≤ 1.0, Delta = 1e-5	If Fed-F1 drop >5%: Collapse to 2 silos; isolate limitation.
Phase 4: Causal Explanations	Extract GNNExplainer visual heatmaps.	Subgraph Size ≤ 15 nodes	If slow generation: Move fidelity scoring to Phase 6.
Phase 5: Agentic SAR	Automate CrewAI reporting from Layer 4 subgraphs.	Report Hallucination < 2%	Guard script triggers automated re-run upon mismatch.
Phase 6: Dashboard Demo	Deploy FastAPI + Streamlit human-in-the-loop frontend.	Sub-60s e2e execution time.	Downgrade 3D PyVis to static Matplotlib frames if laggy.